

Classification of PIR Sensor Data Using Decision Tree and Random Forest Models

Author: Anthony Mackenzi Fernando

Table of Contentss

1. Abstract
2. Introduction
3. Methodology
 - Data Description
 - Data Cleaning and Preprocessing
 - Exploratory Data Analysis
 - Model Selection and Justification
 - Parameter Tuning
4. Results
5. Discussion
6. Conclusion
7. References

1. Abstract

This report explores the use of supervised machine learning models—specifically Decision Tree and Random Forest classifiers—for classifying human activity using data from a PIR (Passive Infra-Red) sensor system. The dataset contains 7651 rows of data captured every 4 seconds from a PIR sensor node, and includes 55 analog readings, ambient temperature, and class labels. The dataset was preprocessed, and key features were used to train and test models. Results show that the Random Forest classifier outperforms the Decision Tree in accuracy and stability.

2. Introduction

With the proliferation of smart devices and sensor-based technologies, there is a growing demand for intelligent systems capable of analyzing time-series sensor data. In home automation and security systems, detecting human presence and motion efficiently is crucial. This project leverages PIR sensor data to classify human activity using two classification algorithms and compares their effectiveness.

3. Methodology

3.1 Data Description

The PIRvision dataset contains occupancy detection data collected from a Synchronized Low-Energy Electronically-chopped Passive Infra-Red sensing node in residential and office environments. Each observation represents 4 seconds of human activity within the sensor's field of view.

Variable Information:

- **Date:** Date of observation
- **Time:** Time of observation
- **Label:** Activity label (0: Vacancy, 1: Stationary human presence)
- **Temperature_F:** Ambient temperature in Fahrenheit
- **PIR_1 to PIR_55:** 55 raw analog sensor readings

3.2 Data Cleaning and Preprocessing

The `Date` and `Time` columns were combined into a single `Timestamp` and later dropped to retain only numerical features compatible with `scikit-learn`. The dataset contained 7651 rows initially, with 6116 rows remaining after removing duplicates and irrelevant entries. No missing values were found after preprocessing.

```
#combine date & time
df['Timestamp'] = pd.to_datetime(df['Date'] + ' ' + df['Time'])
df = df.drop(columns=['Date', 'Time'])

#find null values
print(df.isnull().sum())
```

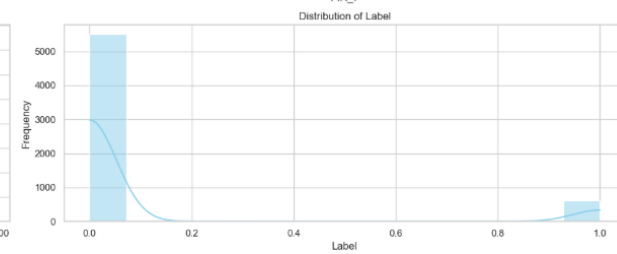
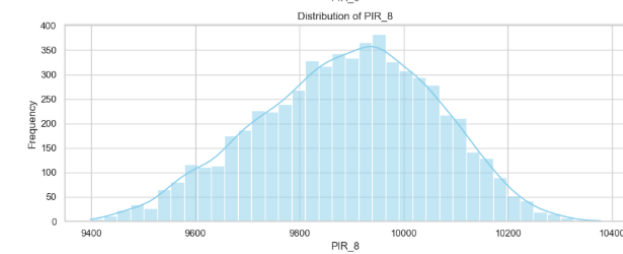
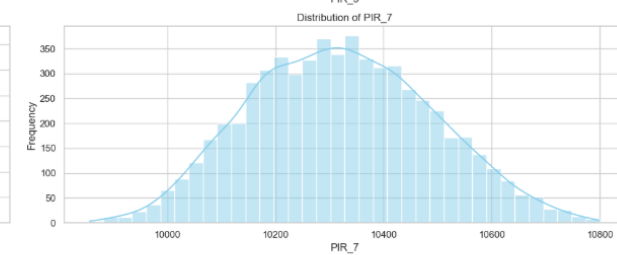
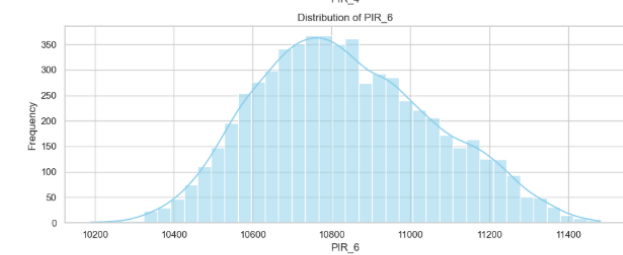
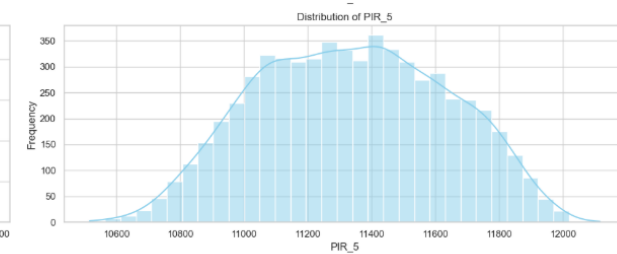
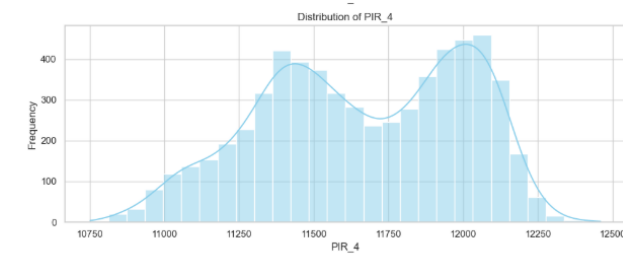
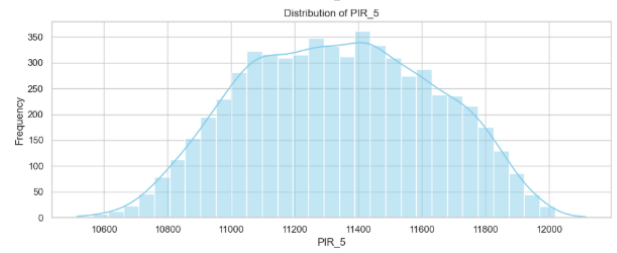
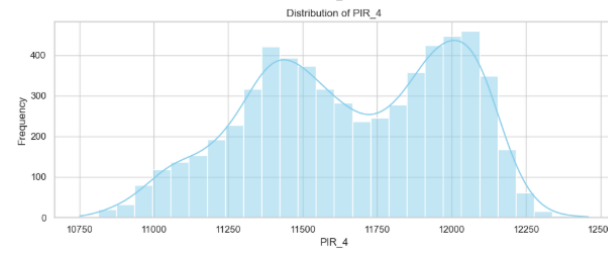
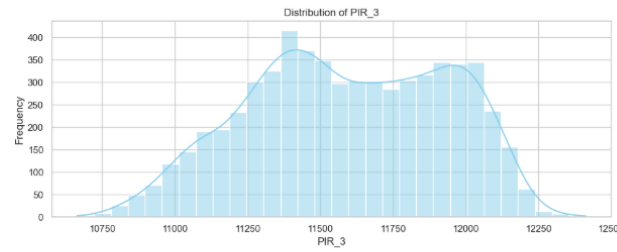
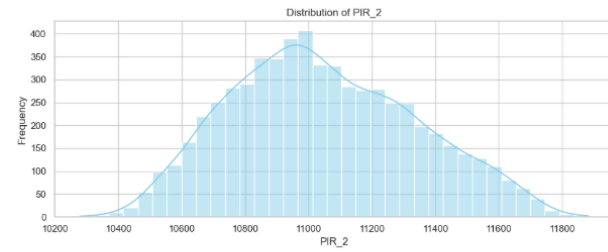
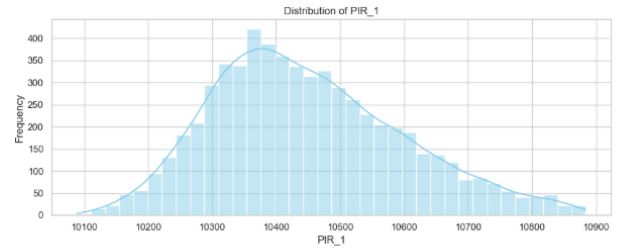
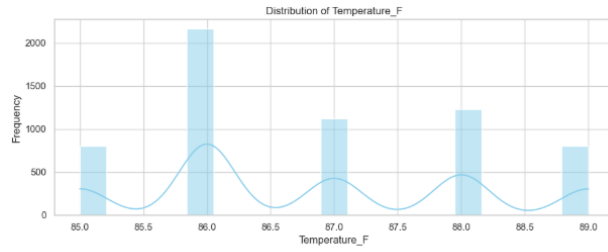
3.3 Exploratory Data Analysis

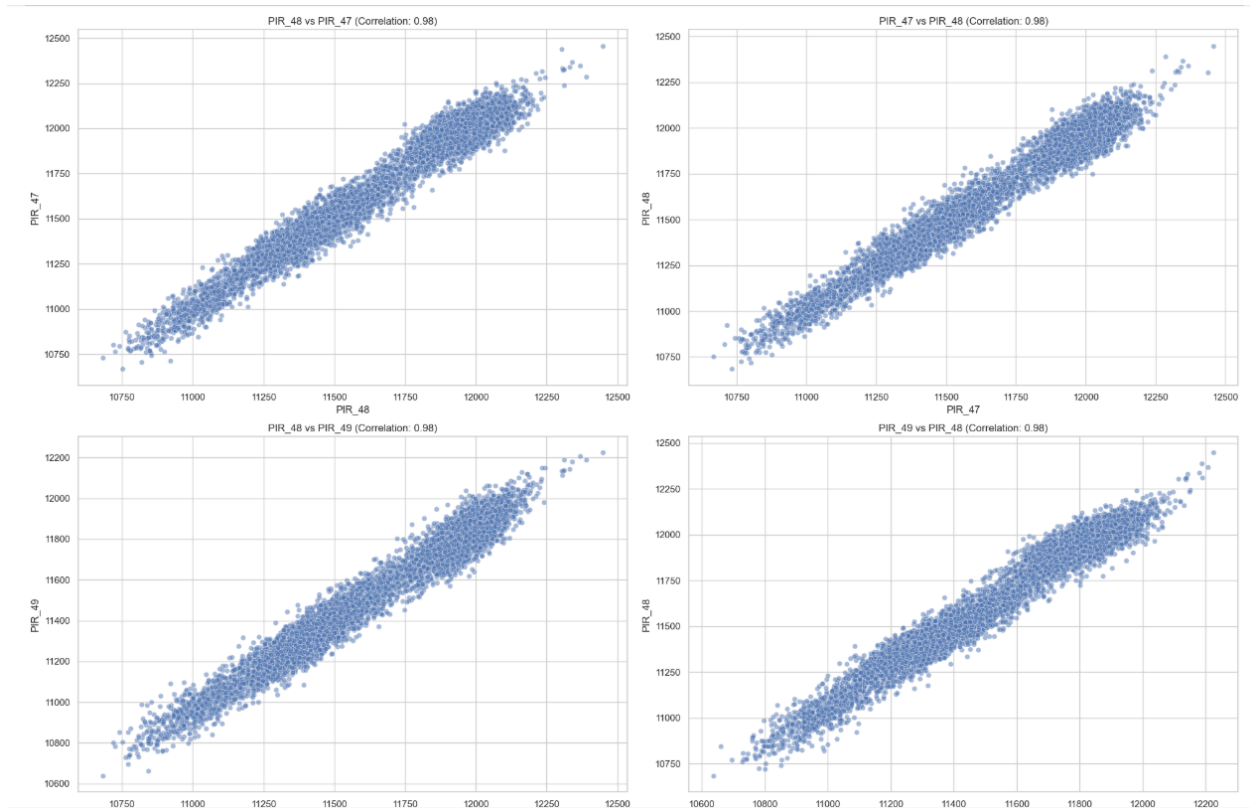
Descriptive statistics revealed the central tendency and spread of each variable. Below is a summary of key variables:

Variable	Mean	Std Dev	Min	25%	50%	75%	Max
Temperature_F	86.85	1.26	85.00	86.00	87.00	88.00	89.00
PIR_1	10449.57	149.16	10089	10339	10431	10545	10884
PIR_2	11048.43	291.68	10279	10832	11022	11260	11883
PIR_3	11592.97	338.62	10664	11339	11589	11884	12413
PIR_4	11664.00	340.47	10752	11399	11663.5	11968	12460
PIR_5	11338.26	295.76	10516	11108	11336	11564	12116
PIR_6	10835.19	222.85	10188	10671	10815	10990	11482
PIR_7	10321.71	167.98	9856	10198	10318	10438	10800
PIR_8	9890.50	169.85	9398	9774	9901	10014	10378
Label	0.10	0.30	0.00	0.00	0.00	0.00	1.00

Top 5 correlated feature pairs were plotted using scatter plots to visualize relationships.

```
plt.figure(figsize=(20, 20))
for i, (col1, col2) in enumerate(top_corr_pairs.index[:5]):
    plt.subplot(3, 2, i + 1)
    sns.scatterplot(x=df_cleaned[col1], y=df_cleaned[col2], alpha=0.5)
    plt.title(f'{col1} vs {col2} (Correlation: {correlation_matrix.loc[col1, col2]:.2f})')
    plt.xlabel(col1)
    plt.ylabel(col2)
plt.tight_layout()
plt.show()
```





3.4 Model Selection and Justification

Two supervised classification models were chosen:

- **Decision Tree Classifier:** Easy to interpret and fast to train.
- **Random Forest Classifier:** More robust and accurate through ensemble learning.

3.5 Parameter Tuning

Hyperparameters were tuned using GridSearchCV with 5-fold cross-validation:

- **Decision Tree:**
 - max_depth: [5, 10, 20, None]
 - min_samples_split: [2, 5, 10]
- **Random Forest:**
 - n_estimators: [50, 100]
 - max_depth: [10, None]
 - min_samples_split: [2, 5]

s4. Results

Sample Data (first 5 rows):

	Date	Time	Label	Temperature_F	PIR_1	PIR_2	...	PIR_55
0	2024-08-08	19:19:56	0	86	10269	10721	...	10329
1	2024-08-08	19:20:12	1	86	10364	10907	...	10357
2	2024-08-08	19:20:28	0	86	10329	10793	...	10265
3	2024-08-08	19:20:44	0	86	10169	10425	...	10463
4	2024-08-08	19:21:00	0	86	10320	10667	...	10407

Decision Tree:

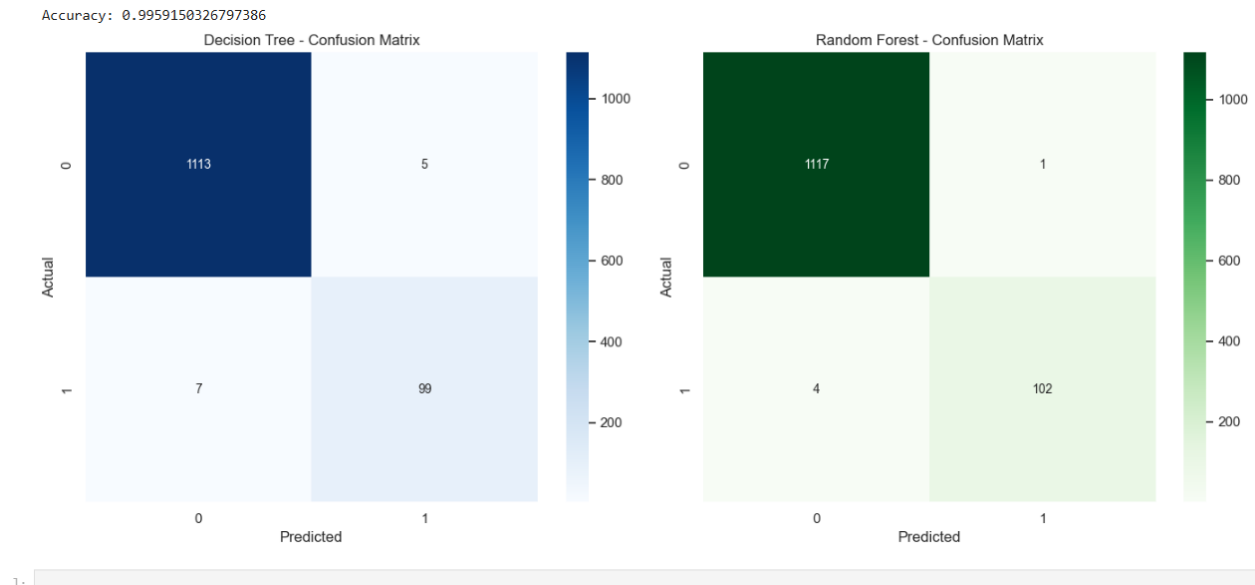
- Best Parameters: {'max_depth': 5, 'min_samples_split': 2}
- Accuracy: 0.9902
- Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1118
1	0.95	0.93	0.94	106
accuracy			0.99	1224
macro avg	0.97	0.96	0.97	1224
weighted avg	0.99	0.99	0.99	1224

Random Forest:

- Best Parameters: {'n_estimators': 50, 'max_depth': 10, 'min_samples_split': 2}
- Accuracy: 0.9959
- Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1118
1	0.99	0.96	0.98	106
accuracy			1.00	1224
macro avg	0.99	0.98	0.99	1224
weighted avg	1.00	1.00	1.00	1224



5. Discussion

The Random Forest classifier significantly outperformed the Decision Tree in terms of accuracy and robustness. This improvement is due to the ensemble method reducing variance and overfitting. Decision Trees, while interpretable, showed slightly poorer generalization on test data. Feature importance plots revealed that only a few PIR sensors contributed dominantly to predictions, suggesting potential for dimensionality reduction.

6. Conclusion

For classifying human activity using PIR sensor data, the Random Forest model is recommended due to its superior performance. It maintains high accuracy and consistency across parameter settings. Future work could include real-time classification, temporal modeling using RNNs, and explainability via SHAP or LIME.

7. References

- Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*.
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- PIRvision Dataset Documentation (2025).
- Lecture notes from [Your Course or Instructor Name] (2025).